

Sistemas Distribuídos e Computação em Nuvem

Aula 4 — Do RPC ao gRPC: chamada remota moderna

C1 · Fundamentos e comunicação distribuída · 27/08/2026

Prof. M.Sc. Howard Cruz Roatti · FAESA · 2026/2

Onde estamos — a trilha do semestre

C1 · Fundamentos e comunicação (Aulas 1–7)

Sockets, concorrência, **gRPC**, REST e IA como serviço.

C2 · Coordenação e consistência (Aulas 8–12)

Mensageria, relógios lógicos, **CAP**, **Raft**, resiliência.

C3 · Nuvem, implantação e segurança (Aulas 13–18)

Containers, nuvem, serverless, observabilidade, segurança.

 Você está na **Aula 4** — subindo o nível de abstração da comunicação.

Retomada — o que você fez em casa

 A aula começa consolidando a entrega da Aula 3.

- Você escreveu o `carga.py` e disparou **N clientes** contra o servidor multicliente?
- O `relatorio_concorrencia.md` mostrou **onde o single-thread trava** conforme a carga cresce?
- Confirmou que **uma thread por cliente** aguenta muito mais que o servidor sequencial?

 Até aqui você trocou **bytes crus** no socket. Hoje vamos fazer uma função em **outra máquina** parecer uma **chamada local**.

Objetivos desta aula

Ao final, você será capaz de:

1. **Explicar** a ideia de **chamada remota de procedimento (RPC)**.
2. **Escrever** um contrato `.proto` com Protocol Buffers.
3. **Implementar** um **servidor** e um **cliente gRPC** em Python.

Conceito 1/3 — A ideia do RPC

- **RPC** (*Remote Procedure Call*) nasceu de um desejo simples: fazer uma função que está em **outra máquina** parecer uma **função local**.
- Em vez de escrever bytes no socket e interpretá-los na mão, você chama `servico.somar(2, 3)` — e o **middleware** (a camada entre a sua aplicação e a rede) **empacota** os argumentos, envia, recebe o resultado e devolve.
- Linha do tempo: **RPC clássico** → **CORBA** → **RMI** (Invocação de Métodos Remotos do Java) → **gRPC** (hoje).

💡 É **transparência de acesso** em ação (Aula 1): usar o remoto **como se fosse local**. A ementa fala de **RPC e RMI** — tratamos ambos como a origem do gRPC.

Conceito 2/3 — O contrato: `.proto` (Protocol Buffers)

- Para trafegar na rede, uma estrutura vira **sequência de bytes** e é remontada do outro lado — isso é a **serialização**.
- No gRPC, o **formato** e o **contrato** do serviço ficam num arquivo `.proto` (**Protocol Buffers**): você declara **quais operações** o serviço oferece e **quais mensagens** trafegam (campos e tipos).
- Uma ferramenta lê o `.proto` e **gera o código** de cliente e servidor — os **stubs**, que escondem a rede e fazem a chamada remota **parecer local**.

 A inversão importante: **o contrato existe ANTES do código**, e os dois lados são **obrigados a respeitá-lo**. Muda o contrato → rege os stubs → os dois lados se atualizam.

Conceito 3/3 — Por que gRPC é usado hoje

Três motivos

1. **Binário compacto** — o Protocol Buffers trafega **muito menos bytes** que JSON/XML.
2. **HTTP/2** — várias chamadas na **mesma conexão**, sem reabrir a cada pedido.
3. **Contrato tipado** — erros de tipo aparecem **cedo** (na geração dos stubs), não em produção.

Onde usar

- **Interno** (serviço ↔ serviço, microserviços de IA): **gRPC** vence.
- **Externo** (público consumindo): **REST** costuma vencer pela **simplicidade** — é a **próxima aula**.

💡 Guarde: **gRPC para dentro, REST para fora**. Seu serviço de IA vai ter as duas interfaces.

Laboratório

Seu primeiro serviço gRPC — passo a passo

Lab · Passo 1 — o contrato `servico.proto`

```
syntax = "proto3";

package calculadora;

service Calculadora {
  rpc Somar (Operandos) returns (Resultado);  // a operação remota
}

message Operandos {           // o que o cliente ENVIA
  int32 a = 1;                // o "= 1" é a posição do campo no formato binário
  int32 b = 2;
}

message Resultado {           // o que o servidor DEVOLVE
  int32 valor = 1;
}
```

💡 Isto é **só o contrato** — nenhuma lógica ainda. Os números (= 1 , = 2) identificam o campo nos bytes, **não** são valores.

Lab · Passo 2 — gerar os stubs

No terminal, na pasta do `.proto`, rode o **protoc** (vem no `grpcio-tools`):

```
pip install grpcio grpcio-tools
python -m grpc_tools.protoc -I. --python_out=. --grpc_python_out=. servico.proto
```

Isso **gera dois arquivos** (não edite!):

- `servico_pb2.py` — as mensagens (`Operandos` , `Resultado`).
- `servico_pb2_grpc.py` — os **stubs** de cliente e servidor.

⚠ Toda vez que você **mudar o** `.proto`, rode este comando de novo para **regenerar** os stubs. É o contrato mandando no código.

Lab · Passo 3 — `servidor_grpc.py`

```
import grpc
from concurrent import futures
import servico_pb2, servico_pb2_grpc

class CalculadoraServicer(servico_pb2_grpc.CalculadoraServicer):
    def Somar(self, request, context):          # implementa a operação
        return servico_pb2.Resultado(valor=request.a + request.b)

servidor = grpc.server(futures.ThreadPoolExecutor(max_workers=10)) # threads da Aula 3!
servico_pb2_grpc.add_CalculadoraServicer_to_server(CalculadoraServicer(), servidor)
servidor.add_insecure_port("127.0.0.1:50051")
servidor.start()
print("[servidor gRPC] ouvindo em 127.0.0.1:50051", flush=True)
servidor.wait_for_termination()
```

💡 Repare no `ThreadPoolExecutor`: o gRPC atende vários clientes com o **pool de threads** que você estudou na Aula 3.

Lab · Passo 4 — `cliente_grpc.py` e rodar

```
import grpc
import servico_pb2, servico_pb2_grpc

with grpc.insecure_channel("127.0.0.1:50051") as canal:
    stub = servico_pb2_grpc.CalculadoraStub(canal)
    resposta = stub.Somar(servico_pb2.Operandos(a=2, b=3))  # parece local!
    print("2 + 3 =", resposta.valor)
```

Dois terminais:

```
python servidor_grpc.py      # Terminal 1 → ouvindo em 50051
python cliente_grpc.py      # Terminal 2 → 2 + 3 = 5
```

💡 `stub.Somar(...)` é uma **chamada de rede** disfarçada de chamada de função. O middleware do gRPC cuidou de tudo.

Lab · Checkpoints & problemas comuns (Windows)

✓ O que você deve ver

- `protoc` cria `servico_pb2.py` e `servico_pb2_grpc.py`.
- Servidor: ouvindo em `127.0.0.1:50051`.
- Cliente: `2 + 3 = 5`.

🔧 Se der erro

- `ModuleNotFoundError: servico_pb2` → você não gerou os stubs (Passo 2) ou está em outra pasta.
- `No module named grpc` → falta `pip install grpcio grpcio-tools`.
- Porta ocupada → troque `50051` por outra.
- Mudou o `.proto` e nada mudou? **Regere os stubs.**

No seu trabalho — C1.A2

- Uma das **duas interfaces** exigidas pelo C1.A2 é **gRPC**. Com o `.proto` e os stubs desta aula, você **implementa o método** do serviço.

💡 Puxa direto para o kit:

- **TAREFA 4** — implementar o método gRPC `PreverLote` e **regerar os stubs**.

Repositório: `sd-2026-2-kit-c1a2` (o `.proto` já está lá, em `proto/`).

Atividade para casa — ampliar o contrato

1. **Adicione um segundo método** ao `servico.proto` — por exemplo `Multiplicar (Operandos) returns (Resultado)`.
2. **Regere os stubs** (Passo 2) e **implemente** o novo método no servidor.
3. **Chame os dois** métodos pelo cliente e confira os resultados.
4. **Escreva um `README.md`** explicando **o que acontece se você mudar o contrato** (renomear um campo, trocar um tipo): por que os **dois lados** precisam reger os stubs.

 **Entregar até a próxima aula:** serviço gRPC com **2 métodos** + `README` sobre o **impacto de mudar o contrato**.

◆ Foco ENADE

O que costuma cair:

- Conceito de **RPC** e **transparência de acesso**.
- **RMI** e invocação de métodos remotos.
- **Serialização** e representação de dados na rede.
- **Middleware** de comunicação e **stubs**.

Termos-chave: RPC · RMI · gRPC · Protocol Buffers · Stub · Serialização · Middleware

💡 A serialização (converter estruturas em bytes) costuma aparecer junto, assim como a comparação **gRPC × outras formas de integração** (REST, mensageria).

Questão de autoavaliação (estilo ENADE)

No gRPC, qual é a função do arquivo com extensão `.proto` ?

- A) Conter a **implementação completa** do servidor.
- B) **Definir o contrato** (mensagens e operações), a partir do qual se **geram os stubs**.
- C) Ser **gerado automaticamente** após a implementação do servidor.
- D) **Eliminar** a necessidade de comunicação em rede.
- E) Armazenar as **credenciais** de autenticação do serviço.

Resolução — alternativa B

- O `.proto` é o **contrato** e vem **antes** do código; a ferramenta **gera**, a partir dele, os **stubs** de cliente e de servidor.
- **C** inverte a ordem (o `.proto` não é gerado — ele **é a origem**).
- **A**, **D** e **E** descrevem coisas que o `.proto` **não** faz (implementação, eliminar a rede, credenciais).

💡 É o seu `servico.proto` de hoje virando questão de prova.

Fora da sala · Glossário

Para estudar

- **Coulouris**, cap. 5 — Invocação remota (RPC/RMI).
- Documentação oficial do **gRPC** (Python) e do **Protocol Buffers**.
- Abra o `proto/` do **kit da C1** e leia o contrato já pronto.

Glossário

- **RPC**: invocar uma função que está em outra máquina.
- **gRPC**: RPC atual, binário, sobre HTTP/2, contrato em `.proto`.
- **Protocol Buffers**: formato binário que define o contrato.
- **Stub**: código gerado que representa o serviço remoto.
- **Middleware**: camada que cuida da comunicação distribuída.

Até a próxima aula 🚀

Entregue o serviço gRPC com 2 métodos + README. A Aula 5 começa revendo isto.

Próxima: Serviços web — REST e OpenAPI com FastAPI (a interface para fora).

 [Voltar ao índice](#)
todas as aulas